

SSGMCE	SHRI SANT GAJANAN MAHARAJ COLLEGE OF ENGG.		LABORATORY MANUAL	
	PRACTICAL EXPERIMENT INSTRUCTION SHEET			
	EXPERIMENT TITLE : WRITE A PROGRAM TO IMPLEMENT A* ALGORITHM USING PYTHON			
EXPERIMENT NO. : SSGMCE/WI/IT/6IT09/04			ISSUE NO. : 00	ISSUE DATE : 06-01-2025
REV. DATE : NA		REV. NO. : 00	DEPTT. : INFORMATION TECHNOLOGY	
LABORATORY : COMPUTER SKILL LAB IV			SEMESTER : VI	PAGE : 1 OF 5

01 AIM : Write A Program To Implement A* Algorithm using Python

02 SCOPE : The experiment will be focusing on understanding the use of a A* Algorithm.

03 THEORY :

The A* search algorithm is a popular pathfinding algorithm used in many applications, including video games, robotics, and route planning. A* uses heuristics to improve the efficiency of the search by prioritizing paths that are likely to be closer to the goal.

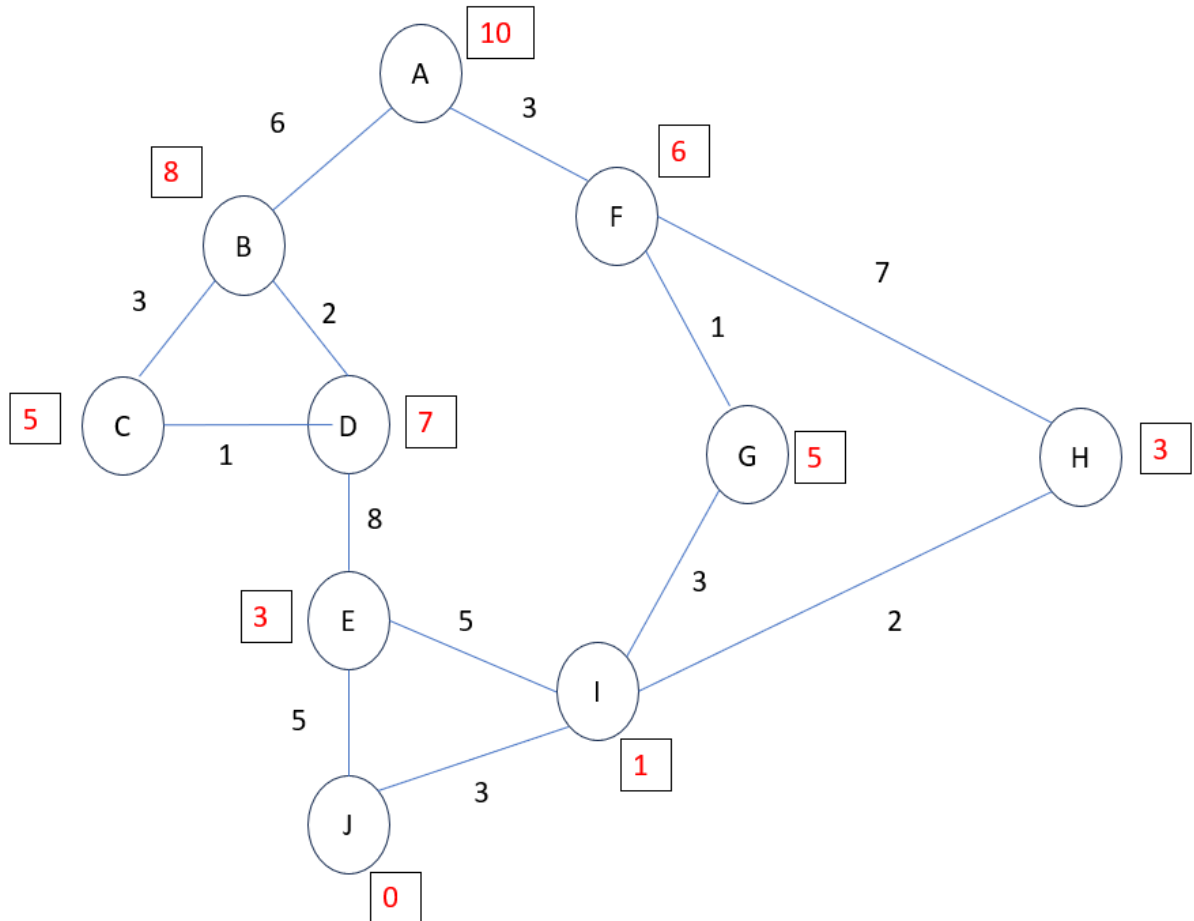
Here's how the A* search algorithm works:

1. Initialization:
 - Start with an open list containing the start node.
 - Start with an empty closed list.
2. While the open list is not empty:
 - Select the node with the lowest f value from the open list. This node is the current node.
 - If the current node is the goal node, the path has been found; reconstruct the path and return it.
 - Move the current node to the closed list.
 - For each neighbor of the current node:
 - If the neighbor is in the closed list or is a wall, skip it.
 - If the neighbor is not in the open list:
 - Compute its g value (cost from the start node to the current node plus the cost from the current node to the neighbor).
 - Compute its h value (heuristic estimate of the cost from the neighbor to the goal).
 - Add it to the open list with $f=g+h$ and set the parent of the neighbor to the current node.
 - If the neighbor is already in the open list:
 - If the new g value is lower than the current g value, update the neighbor's g and f values and update its parent to the current node.
3. If the open list is empty:
 - The goal is unreachable; return failure.

Step-by-step algorithm:

1. Initialize the **priority queue** with the start **node** and its **heuristic value**.
2. Loop until the priority queue is **empty** or the goal node is found:
 - **Pop** the node with the **minimum f-value ($f = g + h$)** from the priority queue.
 - If the popped node is the goal node, **stop** the search.
 - Otherwise, **expand** the node and **update** the priority queue with the neighboring nodes and their **f-values**.
3. If the goal node is found, **reconstruct** the path.

Let us consider the below example:



Weighted Graph

Consider the start node as A and the goal node as J for the above graph so as to analyze the optimal path found by the A* Algorithm.

04 PROGRAM

```
Graph_nodes = {
    'A': [('B', 6), ('F',3)],
    'B': [('C', 3), ('D',2)],
    'C': [('D', 1), ('E',5)],
    'D': [('C', 1), ('E',8)],
    'E': [('I', 5), ('J',5)],
    'F': [('G', 1), ('H',7)],
    'G': [('I', 3)],
    'H': [('I', 2)],
    'I': [('E', 5), ('J',3)],
}

def get_neighbors(v):
    if v in Graph_nodes:
        return Graph_nodes[v]
    else:
        return None

def h(n):
    H_dist = {
        'A': 10,
        'B': 8,
        'C': 5,
        'D': 7,
        'E': 3,
        'F': 6,
        'G': 5,
        'H': 3,
        'I': 1,
        'J': 0,
    }
    return H_dist[n]

def aStarAlgo(start_node, stop_node):

    open_set = set(start_node)
    closed_set = set()
    g = {}
    parents = {}
    g[start_node] = 0
    parents[start_node] = start_node
```

```
while len(open_set) > 0:
    n = None

    for v in open_set:
        if n == None or g[v] + h(v) < g[n] + h(n):
            n = v

    if n == stop_node or Graph_nodes[n] == None:
        pass
    else:
        for (m, weight) in get_neighbors(n):
            if m not in open_set and m not in closed_set:
                open_set.add(m)
                parents[m] = n
                g[m] = g[n] + weight

            else:
                if g[m] > g[n] + weight:
                    g[m] = g[n] + weight
                    parents[m] = n
                    if m in closed_set:
                        closed_set.remove(m)
                    open_set.add(m)

    if n == None:
        print('Path does not exist')
        return None
    if n == stop_node:
        path = []

        while parents[n] != n:
            path.append(n)
            n = parents[n]

        path.append(start_node)

        path.reverse()

        print('Path Found: {}'.format(path))
        return path
    open_set.remove(n)
    closed_set.add(n)

print('Path does not exist')
return None
```

```
aStarAlgo('A', 'J')
```

05 OUTPUT

Path found: ['A', 'F', 'G', 'I', 'J']

06 CONCLUSION

This is how we have successfully implemented the A* algorithm to find the optimal path using heuristic value.

PREPARED BY: Prof. S. S. Muddalkar

APPROVED BY:(H.O.D.): Dr. A. S. Manekar